

Resistance PEX Benchmark: kpex vs MAGIC

Comparative analysis of parasitic resistance extraction between:

- **klayout-pex (kpex)** — KLayout-based 2.5D extraction
- **MAGIC** — VLSI layout tool extraction

Compares equivalent INPUT→OUTPUT resistance across via-chain structures parameterised by layer count (L), plane width (W), and via count (V).

```
In [1]: import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scienceplots # noqa: F401
import seaborn as sns

plt.style.use(["science", "ieee", "no-latex"])
sns.set_palette("colorblind")
```

1. Load Data

```
In [2]: PEX_DIR = Path("dataset/resistance/pex")

TOOLS = {
    "kpex": "kpex_resistance.json",
    "magic": "magic_resistance.json",
}

def load_json(name):
    p = PEX_DIR / name
    return json.loads(p.read_text()) if p.exists() else {}

res_data = {t: load_json(f) for t, f in TOOLS.items()}

print("Cells per tool (resistance):")
for t, d in res_data.items():
    print(f" {t}: {len(d)}")
```

```
Cells per tool (resistance):
kpex: 81
magic: 81
```

2. Build Comparison DataFrame

```
In [3]: def parse_res_name(name):
        """Extract PDK and parameters from resistance GDS filename.

        Format: res_{pdk}_L{layers}_W{width}_V{vias}.gds
        """
        stem = name.replace(".gds", "")
        parts = stem.split("_")
        pdk = parts[1]
        params = {p[0]: int(p[1:]) for p in parts[2:]}
        return pdk, params["L"], params["W"], params["V"]

all_cells = set(res_data["kpex"]) & set(res_data["magic"])
print(f"Cells common to both tools: {len(all_cells)}")

rows = []
for cell in sorted(all_cells):
    pdk, layers, width, vias = parse_res_name(cell)
    for tool in TOOLS:
        entry = res_data[tool].get(cell)
        if entry is None:
            continue
        rows.append({
            "cell": cell,
            "pdk": pdk,
            "layers": layers,
            "width_um": width,
            "vias": vias,
            "tool": tool,
            "resistance_ohm": entry["resistance_ohm"],
        })

df = pd.DataFrame(rows)
df.head(10)
```

Cells common to both tools: 81

```
Out[3]:
```

	cell	pdk	layers	width_um	vias	tool	resistance_ohm
0	res_gf180_L2_W10_V1.gds	gf180	2	10	1	kpex	6.285124
1	res_gf180_L2_W10_V1.gds	gf180	2	10	1	magic	4.639810
2	res_gf180_L2_W10_V3.gds	gf180	2	10	3	kpex	2.103917
3	res_gf180_L2_W10_V3.gds	gf180	2	10	3	magic	1.587600
4	res_gf180_L2_W10_V5.gds	gf180	2	10	5	kpex	1.267840
5	res_gf180_L2_W10_V5.gds	gf180	2	10	5	magic	1.001190
6	res_gf180_L2_W1_V1.gds	gf180	2	1	1	kpex	6.285124
7	res_gf180_L2_W1_V1.gds	gf180	2	1	1	magic	4.590450
8	res_gf180_L2_W1_V3.gds	gf180	2	1	3	kpex	3.145619
9	res_gf180_L2_W1_V3.gds	gf180	2	1	3	magic	2.285760

3. Resistance Correlation — Tool Pairs

```
In [4]: res_pivot = df.pivot_table(
        index=["cell", "pdk", "layers", "width_um", "vias"],
        columns="tool",
        values="resistance_ohm",
    ).reset_index()

pdk_s = sorted(df["pdk"].unique())
colors = sns.color_palette("colorblind", n_colors=len(pdk_s))
pdk_colors = dict(zip(pdk_s, colors))

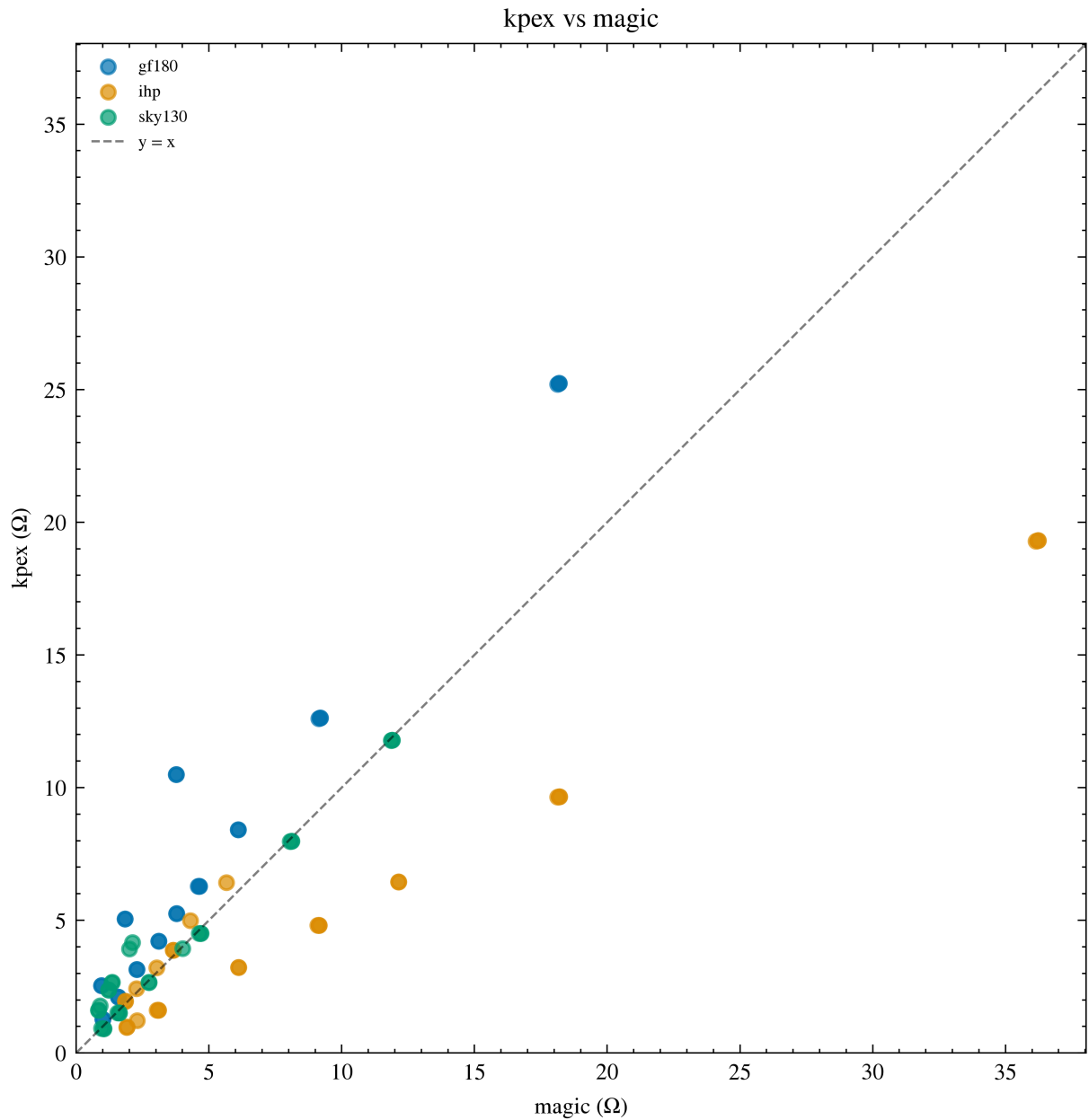
tool_pairs = [("kpex", "magic")]

fig, ax = plt.subplots(figsize=(7, 6))

t_y, t_x = "kpex", "magic"
for pdk in pdk_s:
    mask = res_pivot["pdk"] == pdk
    subset = res_pivot[mask]
    ax.scatter(subset[t_x], subset[t_y],
               color=pdk_colors[pdk], alpha=0.7, s=25, label=pdk)
vals = res_pivot[[t_y, t_x]].values.flatten()
lims = [0, np.nanmax(vals) * 1.05]
ax.plot(lims, lims, "k--", lw=0.8, alpha=0.5, label="y = x")
ax.set_xlim(lims)
ax.set_ylim(lims)
ax.set_xlabel(f"{t_x} (Ω)")
ax.set_ylabel(f"{t_y} (Ω)")
ax.set_title(f"{t_y} vs {t_x}")
ax.legend(fontsize=6)
ax.set_aspect("equal")

fig.suptitle("Equivalent Resistance – Tool Correlation", y=1.02)
fig.tight_layout()
plt.show()
```

Equivalent Resistance — Tool Correlation



```
In [5]: for t_a, t_b in tool_pairs:
        valid = res_pivot[[t_a, t_b]].dropna()
        valid = valid[valid[t_b] > 0]
        rel_err = (valid[t_a] - valid[t_b]) / valid[t_b] * 100
        print(f"{t_a} vs {t_b}:")
        print(f"  Mean relative error: {rel_err.mean():.1f}%")
        print(f"  Median relative error: {rel_err.median():.1f}%")
        print(f"  Std: {rel_err.std():.1f}%")
        print(f"  Range: [{rel_err.min():.1f}%, {rel_err.max():.1f}%]")
        print()
```

kpex vs magic:

Mean relative error: 21.7%
 Median relative error: 6.0%
 Std: 60.8%
 Range: [-49.0%, 178.3%]

4. Relative Error Distribution

```
In [6]: layer_vals = sorted(df["layers"].unique())
layer_colors = dict(zip(layer_vals, sns.color_palette("colorblind", n_colors)))

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

for row, (t_a, t_b) in enumerate(tool_pairs):
    err_col = f"err_{t_a}_vs_{t_b}"
    mask = res_pivot[[t_a, t_b]].notna().all(axis=1) & (res_pivot[t_b] > 0)
    res_pivot.loc[mask, err_col] = (
        (res_pivot.loc[mask, t_a] - res_pivot.loc[mask, t_b])
        / res_pivot.loc[mask, t_b] * 100
    )

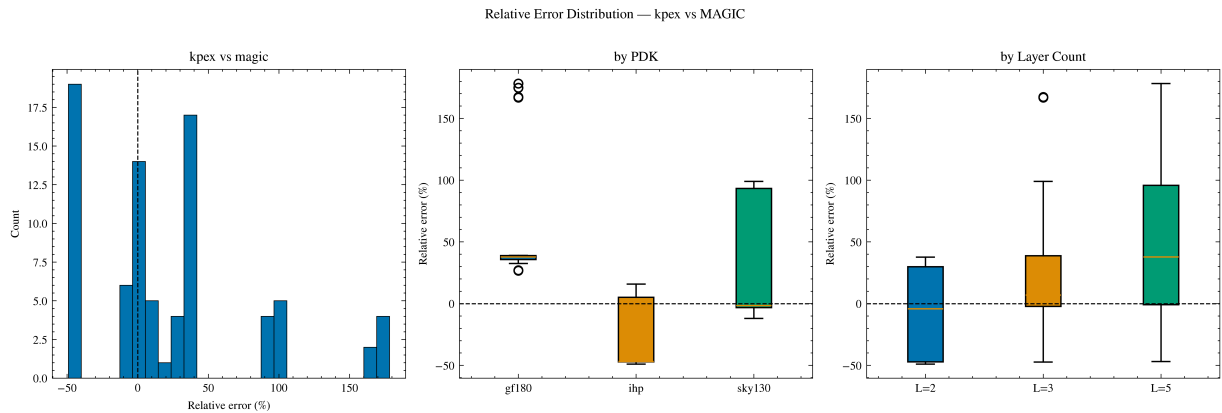
    ax = axes[0]
    ax.hist(res_pivot[err_col].dropna(), bins=25, edgecolor="black", linewidth=0.8)
    ax.axvline(0, color="k", ls="--", lw=0.8)
    ax.set_xlabel("Relative error (%)")
    ax.set_ylabel("Count")
    ax.set_title(f"{t_a} vs {t_b}")

    ax = axes[1]
    data_by_pdk = [
        res_pivot.loc[res_pivot["pdk"] == pdk, err_col].dropna()
        for pdk in pdks
    ]
    bp = ax.boxplot(data_by_pdk, tick_labels=pdks, patch_artist=True)
    for patch, pdk in zip(bp["boxes"], pdks):
        patch.set_facecolor(pdk_colors[pdk])
    ax.axhline(0, color="k", ls="--", lw=0.8)
    ax.set_ylabel("Relative error (%)")
    ax.set_title("by PDK")

    ax = axes[2]
    data_by_layer = [
        res_pivot.loc[res_pivot["layers"] == l, err_col].dropna()
        for l in layer_vals
    ]
    bp = ax.boxplot(data_by_layer, tick_labels=[f"L={l}" for l in layer_vals])
    for patch, l in zip(bp["boxes"], layer_vals):
        patch.set_facecolor(layer_colors[l])
    ax.axhline(0, color="k", ls="--", lw=0.8)
    ax.set_ylabel("Relative error (%)")
    ax.set_title("by Layer Count")

fig.suptitle("Relative Error Distribution - kpex vs MAGIC", y=1.01)
```

```
fig.tight_layout()
plt.show()
```



5. Resistance vs Parameters

```
In [7]: fig, axes = plt.subplots(1, 3, figsize=(12, 4))

width_vals = sorted(df["width_um"].unique())
via_vals = sorted(df["vias"].unique())
tool_markers = {"kpex": "o", "magic": "s"}

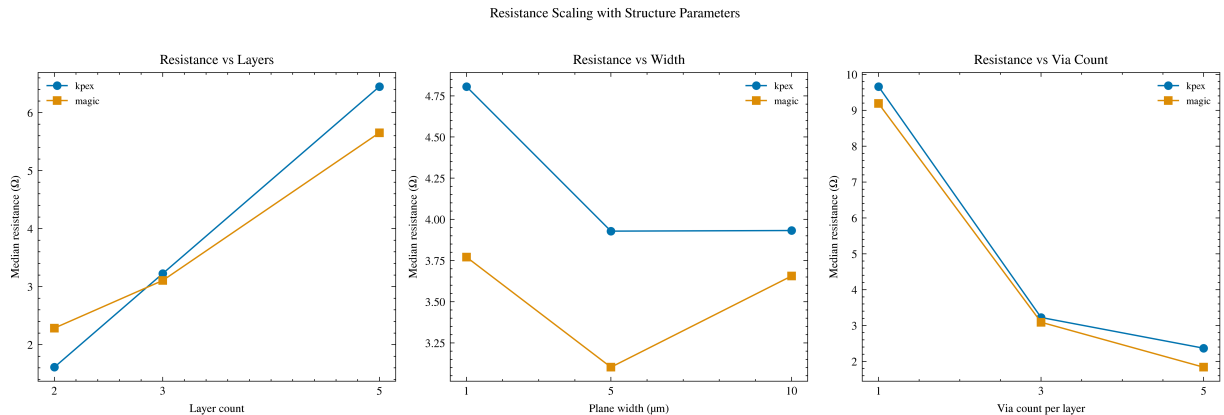
# Resistance vs layer count
ax = axes[0]
for tool in TOOLS:
    grp = df[df["tool"] == tool].groupby("layers")["resistance_ohm"].median()
    ax.plot(grp.index, grp.values, f"{tool_markers[tool]}-", label=tool, mar
ax.set_xlabel("Layer count")
ax.set_ylabel("Median resistance ( $\Omega$ )")
ax.set_title("Resistance vs Layers")
ax.set_xticks(layer_vals)
ax.legend(fontsize=7)

# Resistance vs width
ax = axes[1]
for tool in TOOLS:
    grp = df[df["tool"] == tool].groupby("width_um")["resistance_ohm"].median()
    ax.plot(grp.index, grp.values, f"{tool_markers[tool]}-", label=tool, mar
ax.set_xlabel("Plane width ( $\mu\text{m}$ )")
ax.set_ylabel("Median resistance ( $\Omega$ )")
ax.set_title("Resistance vs Width")
ax.set_xticks(width_vals)
ax.legend(fontsize=7)

# Resistance vs via count
ax = axes[2]
for tool in TOOLS:
    grp = df[df["tool"] == tool].groupby("vias")["resistance_ohm"].median()
    ax.plot(grp.index, grp.values, f"{tool_markers[tool]}-", label=tool, mar
ax.set_xlabel("Via count per layer")
ax.set_ylabel("Median resistance ( $\Omega$ )")
ax.set_title("Resistance vs Via Count")
ax.set_xticks(via_vals)
```

```
ax.legend(fontsize=7)

fig.suptitle("Resistance Scaling with Structure Parameters", y=1.02)
fig.tight_layout()
plt.show()
```



6. Per-PDK Detailed Comparison

```
In [8]: for pdk in pdks:
        subset = res_pivot[res_pivot["pdk"] == pdk].copy()
        if subset.empty:
            continue

        fig, axes = plt.subplots(1, 3, figsize=(14, 4))

        # Grouped bar chart: resistance by layer count and via count
        ax = axes[0]
        pdk_df = df[df["pdk"] == pdk]
        bar_data = pdk_df.pivot_table(
            index=["layers", "vias"], columns="tool", values="resistance_ohm",
        ).reset_index()
        bar_data["label"] = bar_data.apply(lambda r: f"L{int(r['layers'])}V{int(r['vias'])}",
            axis=1)
        x = np.arange(len(bar_data))
        tool_list = [t for t in ["kpex", "magic"] if t in bar_data.columns]
        n_tools = len(tool_list)
        w = 0.8 / n_tools
        for i, tool in enumerate(tool_list):
            offset = (i - (n_tools - 1) / 2) * w
            ax.bar(x + offset, bar_data[tool], w, label=tool, alpha=0.8)
        ax.set_xticks(x)
        ax.set_xticklabels(bar_data["label"], rotation=45, ha="right", fontsize=7)
        ax.set_ylabel("Resistance (Ω)")
        ax.set_title(f"Mean Resistance")
        ax.legend(fontsize=7)

        # Relative error vs via count
        ax = axes[1]
        for t_a, t_b in tool_pairs:
            err_col = f"err_{t_a}_vs_{t_b}"
            if err_col in subset.columns:
                ax.scatter(subset["vias"], subset[err_col],
```

```

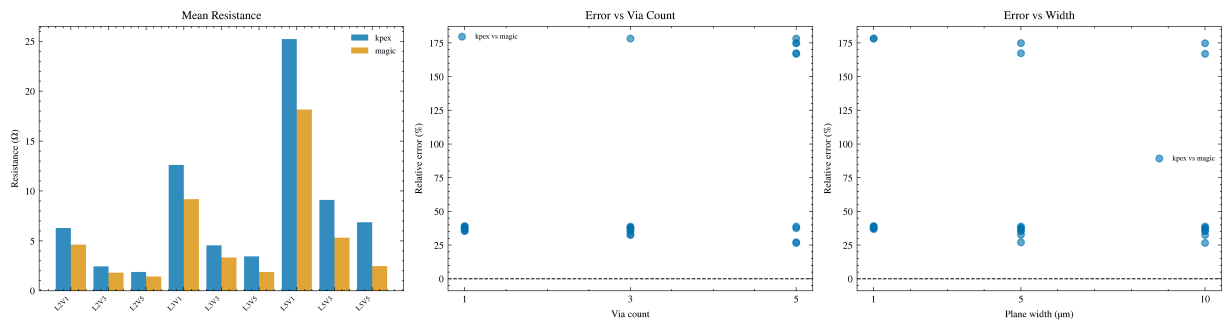
s=30, alpha=0.6, label=f"{t_a} vs {t_b}")
ax.axhline(0, color="k", ls="--", lw=0.8)
ax.set_xlabel("Via count")
ax.set_ylabel("Relative error (%)")
ax.set_title("Error vs Via Count")
ax.set_xticks(via_vals)
ax.legend(fontsize=6)

# Relative error vs width
ax = axes[2]
for t_a, t_b in tool_pairs:
    err_col = f"err_{t_a}_vs_{t_b}"
    if err_col in subset.columns:
        ax.scatter(subset["width_um"], subset[err_col],
                    s=30, alpha=0.6, label=f"{t_a} vs {t_b}")
ax.axhline(0, color="k", ls="--", lw=0.8)
ax.set_xlabel("Plane width (μm)")
ax.set_ylabel("Relative error (%)")
ax.set_title("Error vs Width")
ax.set_xticks(width_vals)
ax.legend(fontsize=6)

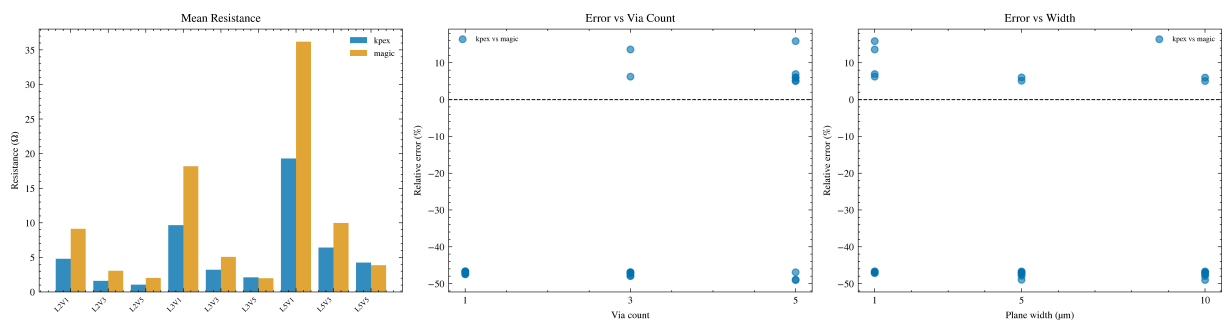
fig.suptitle(f"{pdk} — Resistance Comparison", y=1.02)
fig.tight_layout()
plt.show()

```

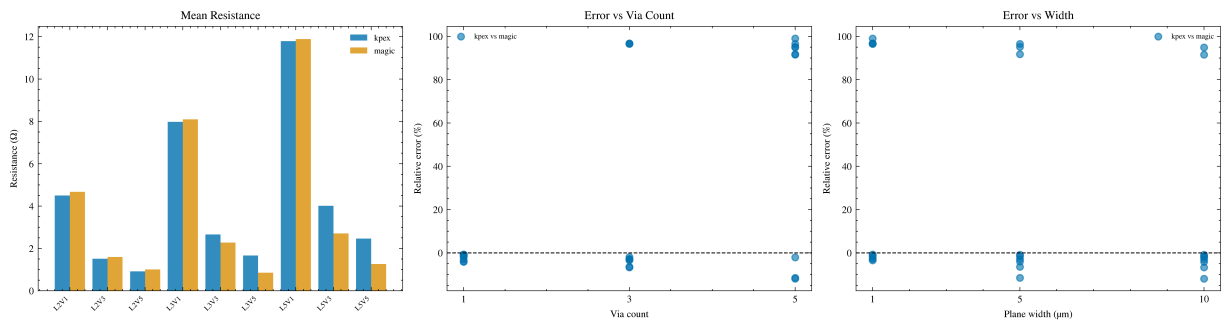
gf180 — Resistance Comparison



ihp — Resistance Comparison



sky130 — Resistance Comparison



7. Summary Statistics

```
In [9]: from scipy import stats

res_summary = df.groupby("tool")["resistance_ohm"].agg(
    ["count", "mean", "median", "std", "min", "max"]
)
res_summary.columns = ["cells", "mean (Ω)", "median (Ω)", "std (Ω)", "min (Ω)", "max (Ω)"]
print("Resistance summary per tool:")
print(res_summary.to_string())
print()

# Per-pair relative error summary and correlation
for t_a, t_b in tool_pairs:
    valid = res_pivot[[t_a, t_b]].dropna()
    valid = valid[valid[t_b] > 0]
    rel_err = (valid[t_a] - valid[t_b]) / valid[t_b] * 100
    r, p = stats.pearsonr(valid[t_a], valid[t_b])
    print(f"{t_a} vs {t_b}:")
    print(f"    Pearson r = {r:.4f} (p = {p:.2e})")
    print(f"    Relative error per PDK:")
    for pdk in pdks:
        pdk_mask = res_pivot["pdk"] == pdk
        pdk_valid = res_pivot.loc[pdk_mask, [t_a, t_b]].dropna()
        pdk_valid = pdk_valid[pdk_valid[t_b] > 0]
        if pdk_valid.empty:
            continue
        errs = (pdk_valid[t_a] - pdk_valid[t_b]) / pdk_valid[t_b] * 100
        print(f"    {pdk:>6s}: mean={errs.mean():+.1f}% median={errs.median():+.1f}%")
    print()
```

Resistance summary per tool:

	cells	mean (Ω)	median (Ω)	std (Ω)	min (Ω)	max (Ω)
tool						
kpex	81	6.014700	4.210144	5.700330	0.914697	25.236787
magic	81	6.375869	3.655763	7.596706	0.839590	36.229100

kpex vs magic:

Pearson r = 0.8168 (p = 1.46e-20)

Relative error per PDK:

gf180: mean=+66.5% median=+37.9% std=58.3%

ihp: mean=-30.9% median=-46.9% std=25.9%

sky130: mean=+29.3% median=-1.6% std=47.8%